

torch.istft#

Description:

Inverse short time Fourier Transform. This is expected to be the inverse of `stft()`. From version 2.1, a warning will be provided if a window is not specified. In a future release, this attribute will be required. Please provide the same window used in the `stft` call. It has the same parameters (+ additional optional parameter of length) and it should return the least squares estimation of the original signal. The algorithm will check using the NOLA condition (nonzero overlap). Important consideration in the parameters window and center so that the envelope created by the summation of all the windows is never zero at certain point in time. Specifically, $\sum_{t=-\infty}^{\infty} |w|^{2[n-t \times \text{hop_length}]} = 0$

Function Signature

```
torch.istft(input, n_fft, hop_length=None, win_length=None,
window=None, center=True, normalized=False, onesided=None,
length=None, return_complex=False) → Tensor:#
```

Parameters

Returns

Least squares estimation of the original signal of shape (B?, length) where

Return type

Returns:

Tensor

Notes & Warnings

WARNING

Warning From version 2.1, a warning will be provided if a window is not specified. In a future release, this attribute will be required. Please provide the same window used in the stft call.

Detailed Documentation

Documentation

Inverse short time Fourier Transform. This is expected to be the inverse of `stft()`.

From version 2.1, a warning will be provided if a window is not specified. In a future release, this attribute will be required. Please provide the same window used in the `stft` call.

It has the same parameters (+ additional optional parameter of `length`) and it should return the least squares estimation of the original signal. The algorithm will check using the NOLA condition (nonzero overlap).

Important consideration in the parameters `window` and `center` so that the envelope created by the summation of all the windows is never zero at certain point in time. Specifically, $\sum_{t=-\infty}^{\infty} |w| \cdot 2[n-t \times \text{hop_length}] = 0$ $\sum_{t=-\infty}^{\infty} |w|^2[n-t \times \text{hop_length}] \neq 0$.

Since `stft()` discards elements at the end of the signal if they do not fit in a frame, `istft` may return a shorter signal than the original signal (can occur if `center` is `False` since the signal isn't padded). If `length` is given in the arguments and is longer than expected, `istft` will pad zeros to the end of the returned signal.

If `center` is `True`, then there will be padding e.g. 'constant', 'reflect', etc. Left padding can be trimmed off exactly because they can be calculated but right padding cannot be calculated without additional information.

Example: Suppose the last window is: [17, 18, 0, 0, 0] vs [18, 0, 0, 0, 0]

The `n_fft`, `hop_length`, `win_length` are all the same which prevents the calculation of right padding. These additional values could be zeros or a reflection of the signal so providing `length` could be useful. If `length` is `None` then padding will be aggressively removed (some loss of signal).

[1] D. W. Griffin and J. S. Lim, "Signal estimation from modified short-time Fourier transform," IEEE Trans. ASSP, vol.32, no.2, pp.236-243, Apr. 1984.

The input tensor. Expected to be in the format of `stft()`, output. That is a complex tensor of shape (B?, N, T) where

B? is an optional batch dimension

N is the number of frequency samples, $(n_fft // 2) + 1$ for onesided input, or otherwise `n_fft`.

T is the number of frames, $1 + \text{length} // \text{hop_length}$ for centered `stft`, or $1 + (\text{length} - n_fft) // \text{hop_length}$ otherwise.

Changed in version 2.0: Real datatype inputs are no longer supported. Input must now have a complex datatype, as returned by `stft(..., return_complex=True)`.

`n_fft` (int) – Size of Fourier transform

`hop_length` (Optional[int]) – The distance between neighboring sliding window frames. (Default: `n_fft // 4`)

`win_length` (Optional[int]) – The size of window frame and STFT filter. (Default: `n_fft`)

`window` (Optional[torch.Tensor]) – The optional window function. Shape must be 1d and $\leq n_fft$ (Default: `torch.ones(win_length)`)

`center` (bool) – Whether input was padded on both sides so that the `ttt`-th frame is centered at time $t \times \text{hop_length}$. (Default: True)

`normalized` (bool) – Whether the STFT was normalized. (Default: False)

onesided (Optional[bool]) – Whether the STFT was onesided. (Default: True if $n_fft \neq$ fft_size in the input size)

length (Optional[int]) – The amount to trim the signal by (i.e. the original signal length). Defaults to $(T - 1) * hop_length$ for centered stft, or $n_fft + (T - 1) * hop_length$ otherwise, where T is the number of input frames.

return_complex (Optional[bool]) – Whether the output should be complex, or if the input should be assumed to derive from a real signal and window. Note that this is incompatible with onesided=True. (Default: False)

$B?$ is an optional batch dimension from the input tensor.